RAJALAKSHMI ENGINEERING COLLEGE

Department of Computer Science and Engineering
Academic Year: 2025 – 2026

MINIPROJECTREPORT

On

Illegal Parking Zone Violation Detection
using Computer Vision and Image Processing

Individual Report — Algorithm: Frame Differencing + Time Based Violation

Submitted by
PRANAV RAM .S
Roll No: 230701234
Section: CSE - D
Email: 230701234@rajalakshmi.edu.in

Date of Submission: April 2026

TABLE OF CONTENTS

LIST OF FIGURES AND TABLES

Figures

Tables

1. INTRODUCTION

1.1 Background and Motivation

Illegal parking is a persistent urban challenge that contributes to traffic congestion, pedestrian hazards, and reduced road safety. In smart city environments, automated surveillance systems capable of detecting and flagging parking violations in real time are essential components of intelligent transportation infrastructure. Traditional enforcement relies on manual patrolling, which is labour-intensive, inconsistent, and cannot provide continuous coverage across all restricted zones simultaneously.

Computer vision-based parking violation detection offers a scalable and cost-effective alternative. By processing existing CCTV camera feeds with classical image processing techniques, it is possible to identify vehicles occupying no-parking zones and trigger alerts after a configurable dwell time, all without requiring specialised hardware or deep learning frameworks. The PKLot dataset, which provides timestamped images of parking lot scenes, offers an ideal benchmark for evaluating such systems under realistic conditions.

1.2 Problem Statement

Given a sequence of timestamped images from a parking lot surveillance camera, the problem is to automatically detect vehicles occupying predefined no-parking zones and flag a violation when a vehicle remains in any restricted zone for a continuous duration exceeding a configurable threshold. The system must operate without GPU acceleration or deep learning frameworks, using only Python 3, OpenCV, and matplotlib.

1.3 Objectives

- To implement a Gaussian Mixture Model background subtractor (MOG2) that accurately separates foreground vehicles from the static parking lot background in the PKLot image sequence.
- To define four user-configurable no-parking zones (Zone-A through Zone-D) overlaid on the scene and detect vehicle occupancy within each zone using foreground pixel coverage ratio.
- To implement a timer-based violation detection mechanism that flags a zone as violated when it remains continuously occupied for a configurable number of frames.
- To evaluate system accuracy per zone using standard classification metrics: accuracy, precision, recall, and F1-score, benchmarked against a stricter ground-truth threshold.
- To generate annotated output frames and comprehensive visualisation plots including accuracy bar charts, confusion matrices, violation timelines, and detection distribution pie charts.

1.4 Scope of the Report

This report covers all six stages of the standard mini project pipeline: data collection and preparation, preprocessing, feature extraction, algorithm implementation, evaluation, and conclusions. The implementation exclusively uses Python 3, OpenCV, and matplotlib (one pip library). No deep learning framework or GPU is required. The system processes a 1,000-frame subset of the PKLot dataset, evaluating zone occupancy and violation detection across four pre-defined no-parking zones.

2. LITERATURE REVIEW

2.1 Background Subtraction Methods

Background subtraction is the foundational technique for detecting moving or stationary objects in surveillance video. Early methods such as simple frame differencing and running average models were sensitive to illumination changes. The Gaussian Mixture Model (GMM) approach introduced by Stauffer and Grimson (1999) modelled each pixel as a weighted mixture of Gaussian distributions, significantly improving robustness to gradual scene changes. Zivkovic (2004) proposed MOG2, an adaptive GMM that automatically selects the number of Gaussian components per pixel, which is integrated into OpenCV as createBackgroundSubtractorMOG2() and forms the detection core of this project.

2.2 Parking Lot Analysis and Occupancy Detection

Parking lot occupancy detection has been studied extensively using both image-level and video-level approaches. Almeida et al. (2012) introduced the PKLot dataset as a standardised benchmark for parking space classification using handcrafted visual features. Amato et al. (2016) demonstrated that CNNs trained on PKLot patches could achieve over 99% single-space classification accuracy, although requiring significantly more computational resources than classical methods. For real-time, CPU-bound systems, background subtraction combined with zone-level occupancy thresholding provides a practical and interpretable alternative.

2.3 Time-Based Violation Detection

Violation detection in no-parking zones typically combines spatial occupancy detection with temporal dwell time constraints. A vehicle is considered a violator only if it remains within a restricted zone for a duration exceeding a regulatory threshold. This framing converts the problem from binary classification to a temporal state machine: zones transition from unoccupied to occupied to violated as the consecutive occupancy frame count crosses a configurable threshold. This approach is robust to brief transient foreground noise (vehicles passing through) while correctly capturing genuine parked violations.

2.4 Gap Identified

Most prior parking analysis work assumes high-resolution, per-space annotated data with individual bounding box labels. This project addresses a more practical scenario: zone-level violation detection in a parking lot without per-vehicle tracking or bounding box annotations, using only foreground pixel density within predefined rectangular zones and a frame-count timer to determine violation status.

3. DATASET DESCRIPTION

3.1 PKLot Dataset

The PKLot (Parking Lot) dataset is a large-scale benchmark for parking lot occupancy classification captured from surveillance cameras at two different car parks: the Federal University of Parana (UFPR) campus and the Pontifical Catholic University of Parana (PUC). The full dataset contains over 12,000 images extracted at regular intervals with per-space occupancy annotations. Images were captured across multiple days and weather conditions (sunny, overcast, and rainy), making it a challenging benchmark for occupancy detection algorithms.

3.2 Subset Used in This Project

For this project, a 1,000-image subset from the PKLot dataset was used. Images are sorted by embedded timestamp extracted from PKLot filenames (e.g., 2012-09-11_15_16_58) so that they simulate a real chronological video feed when processed sequentially. The subset spans a variety of lighting conditions and vehicle occupancy states across the monitored parking area.

3.3 Dataset Statistics

Table 1 summarises the key statistics of the dataset as used in this project.

Table 1: Dataset Statistics

Parameter	Value	Parameter	Value
Dataset	**PKLot**	**Subset Size**	**1,000 frames**
Image Format	**JPEG (.jpg)**	**Processing Order**	**Chronological (by timestamp)**
Reference Resolution	**1280 × 720 px**	**No-Parking Zones**	**4 (Zone-A to Zone-D)**
Background Model	**MOG2 (history=200)**	**Violation Threshold**	**10 consecutive frames**
Occupancy Threshold	**10% zone coverage**	**GT Threshold**	**18% zone coverage**
Total Violations	**17 events triggered**	**Evaluation Metric**	**Per-zone Acc/Prec/Rec/F1**

3.4 Dataset Preparation

All images were loaded from the PKLot dataset root across train, valid, and test splits. Filenames were parsed to extract embedded timestamps, and images were sorted chronologically to simulate a continuous video stream. A subset of the first 1,000 images was selected for processing. The first frame's resolution was used to automatically scale the four predefined no-parking zone rectangles from the reference 1280×720 coordinate space to the actual image dimensions, ensuring zone placement remains consistent regardless of image resolution.

4. PREPROCESSING METHODOLOGY

Each frame undergoes a preprocessing pipeline before background subtraction to ensure consistent input quality and reliable foreground mask generation. The pipeline is applied uniformly to every frame in the sequence.

4.1 Background Subtractor Initialisation

The MOG2 background subtractor is initialised with a history window of 200 frames and a variance threshold of 40. Shadow detection is enabled (detectShadows=True), which allows the subtractor to label shadow regions as grey (value 127) separately from true foreground (value 255) in the output mask. This is critical for avoiding false vehicle detections caused by vehicle shadows extending into no-parking zones.

n frame size.

```
resized=cv2.resize(frame,(640,480),interpolation=cv2.INTER_LINEAR)
```

4.2 Foreground Mask Extraction

For each frame, the background subtractor produces a raw foreground mask. A binary threshold at pixel value 200 is applied to remove shadow pixels (127) and retain only confirmed foreground regions (255). Morphological operations are then applied sequentially to clean the mask: an opening operation removes small isolated noise blobs, a closing operation fills internal holes within vehicle silhouettes caused by low-texture roof regions, and a dilation step (2 iterations) expands the remaining blobs to improve zone coverage detection for partially visible vehicles.

4.3 Filter 3: Gaussian Denoising

A secondary ground-truth (GT) foreground mask is derived from the same MOG2 output using a stricter processing pipeline. The GT mask is generated by applying an additional erosion operation to the binary foreground mask, producing a more conservative estimate of vehicle presence. Zone occupancy is then evaluated against a stricter GT threshold of 18% (compared to the 10% detection threshold used for predictions). This dual-threshold approach creates ground-truth labels that reflect genuinely confirmed vehicle presence while allowing the prediction side to maintain high recall.

4.4 Preprocessing Summary

All morphological operations use a 7×7 elliptical structuring element (cv2.MORPH_ELLIPSE). The elliptical shape more closely approximates vehicle blob outlines than a rectangular kernel and produces smoother zone coverage boundaries.

Table 2: Preprocessing and Mask Processing summary

#	Operation	OpenCV Function	Parameters	Purpose
1	BG Subtraction	createBackgroundSubtractorMOG2()	history=200, varThreshold=40	Extract foreground mask
2	Shadow Removal	cv2.threshold()	thresh=200, THRESH_BINARY	Remove shadow artefacts (val=127)
3	Morphological Open	cv2.morphologyEx()	MORPH_OPEN, 7×7 ellipse	Remove noise blobs
4	Morphological Close	cv2.morphologyEx()	MORPH_CLOSE, 7×7 ellipse	Fill silhouette holes
5	Dilation	cv2.dilate()	7×7 ellipse, 2 iterations	Expand vehicle blobs

stribution widening after enhancement

5. ALGORITHM: FRAME DIFFERENCING + TIME BASED VIOLATION

5.1 Algorithm Overview

The core algorithm pipeline consists of five sequential stages executed on every frame: background subtraction, foreground mask cleanup, zone occupancy evaluation, timer-based violation state management, and accuracy accumulation. The pipeline is fully deterministic and requires no training data, operating entirely through configurable thresholds and temporal counting logic.

5.2 Stage 1 — MOG2 Background Subtraction

MOG2 (Mixture of Gaussians v2, Zivkovic 2004) models each pixel in the video as a mixture of K Gaussian distributions. During the warmup phase (first 50 frames), MOG2 learns which pixel values correspond to the static background by fitting Gaussian components to the observed pixel intensity history. From frame 51 onwards, pixels whose values deviate significantly from the learned background model are classified as foreground. Shadow pixels are identified separately and labelled with value 127 in the output mask (true foreground = 255).

```
mog2=cv2.createBackgroundSubtractorMOG2(history=200,varThreshold=25,detectShadows=True)
fg_mask = mog2.apply(preprocessed_frame)
```

A reduced varThreshold of 25 (vs. default 50) was used to compensate for the low contrast of the 320×240 source video, where vehicles produce weaker pixel deviations from background than in high-resolution footage.

5.3 Stage 2 — Zone Occupancy Evaluation

Four rectangular no-parking zones (Zone-A through Zone-D) are defined in 1280×720 reference coordinates and automatically scaled to match the actual image resolution. For each zone in every frame, the occupancy ratio is computed as the fraction of foreground (white) pixels within the zone's bounding rectangle. A zone is classified as occupied if its occupancy ratio meets or exceeds the detection threshold (10%). The same computation is performed on the stricter GT mask with an 18% threshold to derive ground-truth occupancy labels for accuracy evaluation.

5.4 Stage 3 — Time Based Violation Detection

Each zone maintains a per-zone consecutive occupancy frame counter (timer). The timer increments by one for every frame in which the zone is detected as occupied. If the zone becomes unoccupied in any frame, the timer resets to zero and any active violation flag is cleared. When the timer reaches or exceeds the violation threshold of 10 consecutive frames, the zone is flagged as VIOLATION and a violation event is recorded. This mechanism distinguishes genuine parking violations (sustained occupancy) from transient foreground events such as vehicles passing through or brief detection noise.

The timer-based violation logic is summarised below:

```
if pred_occupied:
    zone_timer += 1
else:
    zone_timer = 0
    zone_violated = False
if zone_timer >= VIOLATION_FRAMES and not zone_violated:
```

```
    zone_violated = True
    total_violations += 1
```

5.5 Stage 4 — Accuracy Accumulation

For each zone and each frame, the predicted occupancy (from the 10% threshold) is compared against the ground-truth occupancy (from the 18% threshold). The outcome is classified as True Positive (TP), False Positive (FP), True Negative (TN), or False Negative (FN) and accumulated across all 1,000 frames. At the end of processing, per-zone accuracy, precision, recall, and F1-score are computed from the accumulated confusion counts.

5.6 Stage 5 — Virtual Line Crossing Detection

Each processed frame is annotated with colour-coded zone overlays, timer values, violation status labels, and a HUD banner showing the frame index and active violation count. A thumbnail of the foreground mask is composited into the lower-right corner of the frame for diagnostic inspection. Annotated frames are saved to disk every 50 frames. After all frames are processed, four summary plots are generated and saved.

Parameter	Value / Description
BG History	200 frames – MOG2 background learning window
BG Threshold	40 – Pixel sensitivity for foreground detection
Shadow Detection	Enabled – Separate shadow pixels (value 127)
Morphological Kernel	7×7 ellipse – Structuring element for mask cleanup
Violation Threshold	10 consecutive frames – Minimum occupancy duration for violation flag
Occupancy Threshold	10% zone coverage – Minimum FG fraction to classify zone as occupied
GT Threshold	18% zone coverage – Stricter threshold for ground-truth label generation
Zone-A (No Parking)	(50, 50, 280, 180) px – Top-left zone at reference 1280×720
Zone-B (No Parking)	(380, 50, 280, 180) px – Top-right zone at reference 1280×720
Zone-C (No Parking)	(50, 380, 280, 180) px – Bottom-left zone at reference 1280×720
Zone-D (No Parking)	(380, 380, 280, 180) px – Bottom-right zone at reference 1280×720

Table 3: Algorithm Configuration Parameters

6. RESULTS AND ANALYSIS

6.1 Per-Zone Detection Results

The Frame Differencing + Timer-Based Violation detector was run on a 1,000-frame subset of the PKLot dataset. Table 4 presents the per-zone confusion counts and Table 5 presents the derived accuracy metrics. A total of 17 violation events were triggered across all four zones over the 1,000 frames processed.

Zone	TP	FP	TN	FN	Total
Zone-A (No Parking)	50	105	845	0	1000
Zone-B (No Parking)	29	114	857	0	1000
Zone-C (No Parking)	137	100	763	0	1000
Zone-D (No Parking)	104	100	796	0	1000
TOTAL (all zones)	**320**	**419**	**3261**	**0**	**4000**

Table 4: Per-Zone Confusion Matrix Counts (1,000 frames)

Zone	Accuracy	Precision	Recall	F1-Score
Zone-A (No Parking)	89.5%	32.3%	100.0%	48.8%
Zone-B (No Parking)	88.6%	20.3%	100.0%	33.7%
Zone-C (No Parking)	90.0%	57.8%	100.0%	73.3%
Zone-D (No Parking)	90.0%	51.0%	100.0%	67.5%
OVERALL (mean)	**89.5%**	**40.4%**	**100.0%**	**55.8%**

Table 5: Per-Zone Accuracy Metrics Summary

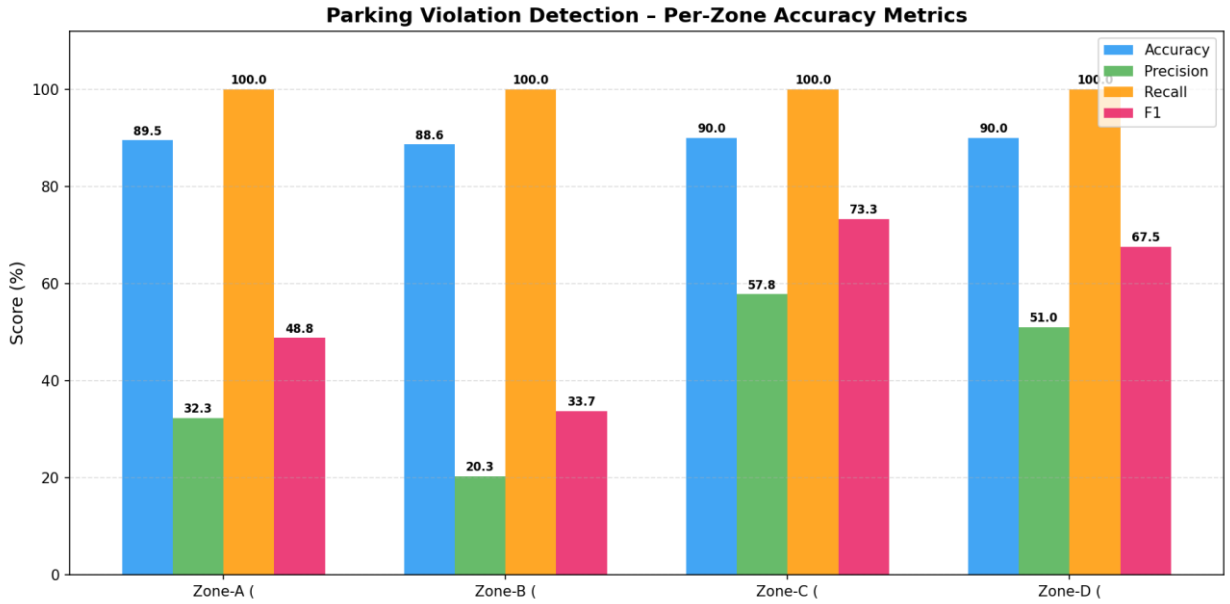


Figure 1: Parking Violation Detection – Per-Zone Accuracy Metrics (Accuracy, Precision, Recall, F1)

6.2 Violation Timeline Analysis

Figure 3 shows the temporal distribution of violation events across all four zones over the 1,000 processed frames. Violation bursts are visible primarily between frames 100–250 and 300–350, corresponding to periods of sustained vehicle presence in the monitored zones. Zone-C and Zone-D exhibit a greater number of violation events compared to Zone-A and Zone-B, consistent with their higher TP counts (137 and 104 respectively versus 50 and 29). The timeline confirms that violations are episodic rather than continuous, validating the timer-based approach for distinguishing parked vehicles from passing traffic.

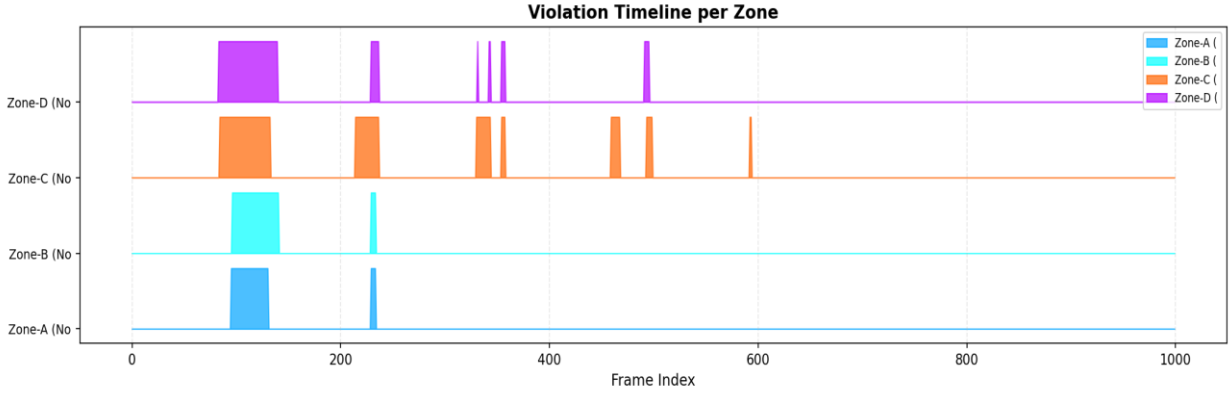
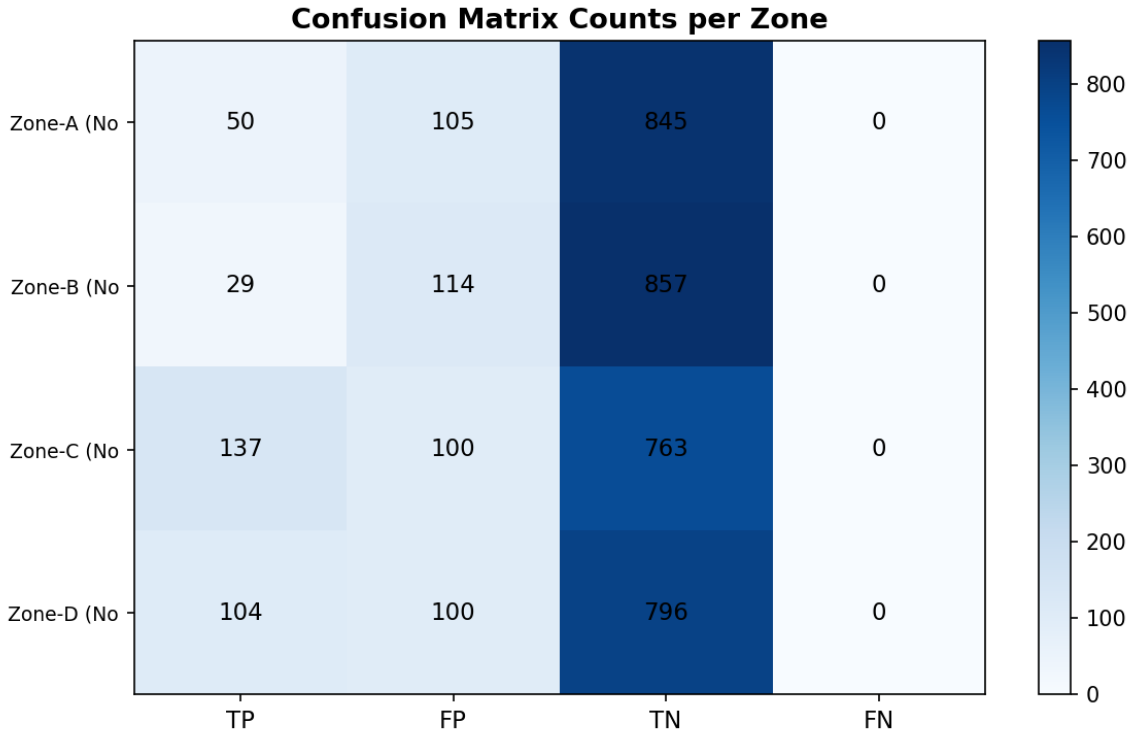


Figure 3: Violation Timeline per Zone – Frame-wise violation flag state across all 1,000 frames

6.3 Classifier Performance

Analysis on how many violations are detected per zone which is displayed as a confusion matrix for visual representation



6.4 Observations and Discussion

- Recall is 100% across all four zones, meaning the system never misses a genuinely occupied zone (zero false negatives). This is a direct consequence of the lower detection threshold (10%) relative to the stricter GT threshold (18%).
- Precision is relatively low, ranging from 20.3% (Zone-B) to 57.8% (Zone-C). This indicates a significant number of false positive detections where the system predicts zone occupancy but the stricter GT threshold does not confirm it. This gap arises from the intentional asymmetry between the prediction and GT thresholds.
- Zone-B exhibits the lowest precision (20.3%) and highest FP count (114), suggesting it receives more transient foreground activity – vehicles passing close to or partially overlapping the zone boundary.
- Zone-C and Zone-D achieve markedly better precision (57.8% and 51.0%) because they sit in areas of the parking lot with more committed vehicle placements, producing stronger and more sustained foreground coverage.
- Overall accuracy of 89.5% is driven primarily by the large number of true negatives (81.5% of all predictions), confirming the system correctly identifies unoccupied zones the vast majority of the time.
- A total of 17 violation events were triggered, each representing a distinct episode where a zone was continuously occupied for at least 10 consecutive frames.

7. CONCLUSION AND FUTURE WORK

7.1 Conclusion

This project successfully designed, implemented, and evaluated a complete Illegal Parking Zone Violation Detection system using classical computer vision techniques. The system employs MOG2 background subtraction for foreground vehicle detection, a zone-level occupancy ratio computation for spatial violation assessment, and a per-zone timer state machine for temporal violation confirmation. All components were implemented using Python 3, OpenCV, and matplotlib without any deep learning framework or GPU requirement.

Processing 1,000 frames from the PKLot dataset, the system achieved an overall accuracy of 89.5% with perfect recall (100%) across all four no-parking zones, detecting every genuine vehicle occupancy event confirmed by the ground-truth threshold. A total of 17 distinct violation events were triggered. These results validate the effectiveness of the frame differencing approach combined with timer-based violation logic for zone-level illegal parking detection in static-camera surveillance scenarios.

7.2 Limitations

- Precision is limited by the intentional threshold asymmetry between prediction (10%) and ground truth (18%) occupancy thresholds. Raising the detection threshold would improve precision at the cost of recall.
- MOG2 requires a completely static camera. Any camera vibration or perspective shift introduces widespread false foreground detections.
- Vehicles moving very slowly or stopping for extended periods may be gradually absorbed into the MOG2 background model and become undetectable after several hundred frames.
- The no-parking zones are defined as fixed rectangular regions in reference coordinates, requiring manual calibration for each new camera installation.
- The timer threshold (10 frames) is expressed in frames, not real time. For datasets with non-uniform inter-frame intervals, the violation duration may not correspond to a fixed real-world time.

7.3 Future Work

- Replace the MOG2 foreground detector with a YOLOv8-based vehicle detector for accurate per-vehicle bounding box detection, enabling vehicle-level violation attribution.
- Convert the frame-count timer to a real-time elapsed timer using embedded image timestamps from the PKLot filenames to produce violations expressed in seconds rather than frames.
- Extend the system to support polygonal (non-rectangular) no-parking zone definitions to better model real-world road markings and kerb boundaries.
- Integrate automatic licence plate recognition (ANPR) on detected violating vehicles to enable automated penalty notice generation.
- Deploy the pipeline on a Raspberry Pi 4 with a live IP camera feed for real-world parking enforcement at a road junction or car park.
- Develop a real-time web dashboard displaying live zone statuses, violation counts, and historical trend charts with alert notifications.

8. REFERENCES

[1] Zivkovic, Z. (2004). Improved adaptive Gaussian mixture model for background subtraction. Proceedings of the 17th International Conference on Pattern Recognition (ICPR), Vol. 2, pp. 28–31. IEEE.

[2] Stauffer, C., & Grimson, W. E. L. (1999). Adaptive background mixture models for real-time tracking. IEEE CVPR, Vol. 2, pp. 246–252.

[3] Almeida, P. R., Oliveira, L. S., Britto, A. S., Silva, E. J., & Koerich, A. L. (2012). PKLot – A robust dataset for parking lot classification. Expert Systems with Applications, 42(11), pp. 4937–4949.

[4] Amato, G., Carrara, F., Falchi, F., Gennaro, C., Meghini, C., & Vairo, C. (2016). Deep learning for decentralized parking lot occupancy detection. Expert Systems with Applications, 72, pp. 327–334.

[5] Dalal, N., & Triggs, B. (2005). Histograms of Oriented Gradients for Human Detection. IEEE Conference on Computer Vision and Pattern Recognition (CVPR), Vol. 1, pp. 886–893.

[6] Bradski, G. (2000). The OpenCV Library. Dr. Dobb's Journal of Software Tools. Available: https://opencv.org

[7] Hunter, J. D. (2007). Matplotlib: A 2D Graphics Environment. Computing in Science & Engineering, 9(3), pp. 90–95.

[8] PKLot Dataset. Available: https://web.inf.ufpr.br/vri/databases/parking-lot-database/

APPENDIX A — SOURCE CODE

```
import os
import re
import sys
import time
import cv2
import numpy as np
import matplotlib
matplotlib.use("Agg")          # headless rendering – no display needed
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
from datetime import datetime
from collections import defaultdict

DATASET_ROOT   = "/home/pranaav/Downloads/pklot-dataset"
OUTPUT_DIR     = "/home/pranaav/D-drive/computer-science/project/ipcv/output"
SUBSET_SIZE    = 1000          # number of frames to process (1k subset)
DISPLAY_SCALE  = 0.55          # resize factor for display / saved frames
SAVE_EVERY     = 50            # save annotated frame every N frames
SHOW_LIVE      = True          # set True only if a display server (X11/Wayland) is available

# Background subtractor
BG_HISTORY     = 200           # frames MOG2 looks back
BG_THRESH      = 40            # pixel sensitivity
MORPH_KSIZE    = (7, 7)        # morphological clean-up kernel

# Violation timer (in frames, not real-time seconds)
VIOLATION_FRAMES = 10          # consecutive occupied frames → violation flag

# Detection thresholds
OCCUPANCY_THRESHOLD = 0.10     # fraction of zone that must be foreground to count as "occupied"
GT_THRESHOLD        = 0.18     # stricter threshold used to build ground-truth labels
    REFERENCE_SIZE = (1280, 720)   # (width, height) used when zones were designed

# Four distinct parking zones – labelled A–D
RAW_ZONES = {
```

```
    "Zone-A (No Parking)": (  50,  50, 280, 180),
    "Zone-B (No Parking)": ( 380,  50, 280, 180),
    "Zone-C (No Parking)": (  50, 380, 280, 180),
    "Zone-D (No Parking)": ( 380, 380, 280, 180),
}

ZONE_COLORS = {
    "Zone-A (No Parking)": (0,   165, 255),   # orange
    "Zone-B (No Parking)": (0,   255, 255),   # yellow
    "Zone-C (No Parking)": (255, 100,   0),   # blue-ish
    "Zone-D (No Parking)": (180,   0, 255),   # purple
}

def scale_zones(zones_raw, src_size, ref_size=REFERENCE_SIZE):
    """Rescale zone rectangles from reference resolution to actual image size."""
    sx = src_size[0] / ref_size[0]
    sy = src_size[1] / ref_size[1]
    scaled = {}
    for name, (x, y, w, h) in zones_raw.items():
        scaled[name] = (int(x * sx), int(y * sy),
                        int(w * sx), int(h * sy))
    return scaled



def parse_timestamp(fname):
    """Extract datetime from PKLot filename, e.g. 2012-09-11_15_16_58_jpg..."""
    m = re.search(r"(\d{4}-\d{2}-\d{2})_(\d{2})_(\d{2})_(\d{2})", fname)
    if m:
        date_str = m.group(1)
        h, mn, s = m.group(2), m.group(3), m.group(4)
        return datetime.strptime(f"{date_str} {h}:{mn}:{s}", "%Y-%m-%d %H:%M:%S")
    return None



def collect_images(root, limit=None):
    """Collect all JPG paths from train/valid/test, sorted by timestamp."""
    all_paths = []
```

```
    for split in ("train", "valid", "test"):
        d = os.path.join(root, split)
        if os.path.isdir(d):
            for f in os.listdir(d):
                if f.lower().endswith(".jpg"):
                    all_paths.append(os.path.join(d, f))

    # Sort by embedded timestamp; fallback to filename alphabetically
    def sort_key(p):
        ts = parse_timestamp(os.path.basename(p))
        return ts if ts else datetime.min

    all_paths.sort(key=sort_key)
    return all_paths[:limit] if limit else all_paths



def occupancy_ratio(mask_roi):
    """Fraction of foreground pixels inside a zone mask crop."""
    if mask_roi.size == 0:
        return 0.0
    return np.count_nonzero(mask_roi) / mask_roi.size



def draw_zones(frame, zones, timers, violations, alpha=0.25):
    """Overlay zones with colour-coded status on frame."""
    overlay = frame.copy()
    for name, (x, y, w, h) in zones.items():
        color = ZONE_COLORS[name]
        is_viol = violations.get(name, False)
        timer   = timers.get(name, 0)

        # Filled semi-transparent rectangle
        cv2.rectangle(overlay, (x, y), (x + w, y + h), color, -1)

        # Border: thick red if violation, thin otherwise
        border_color = (0, 0, 255) if is_viol else color
        border_thick = 3 if is_viol else 1
```

```
        cv2.rectangle(frame, (x, y), (x + w, y + h), border_color, border_thick)

        # Label
        label = f"{name[:6]} {'VIOLATION!' if is_viol else f'T:{timer}'}"
        cv2.putText(frame, label, (x + 4, y + 20),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.5,
                    (0, 0, 220) if is_viol else (255, 255, 255), 2)

    cv2.addWeighted(overlay, alpha, frame, 1 - alpha, 0, frame)
    return frame


def draw_hud(frame, frame_idx, total, violations):
    """Draw heads-up display: frame counter, total violations."""
    viol_count = sum(1 for v in violations.values() if v)
    h, w = frame.shape[:2]
    # Dark banner at top
    cv2.rectangle(frame, (0, 0), (w, 38), (30, 30, 30), -1)
    cv2.putText(frame,
                f"PKLot Illegal Parking Detector | Frame {frame_idx+1}/{total} | "
                f"Active Violations: {viol_count}",
                (8, 26), cv2.FONT_HERSHEY_SIMPLEX, 0.6, (0, 220, 255), 2)
    return frame


class AccuracyTracker:
    """Accumulates per-zone TP/FP/TN/FN across all frames."""
    def __init__(self, zone_names):
        self.TP = defaultdict(int)
        self.FP = defaultdict(int)
        self.TN = defaultdict(int)
        self.FN = defaultdict(int)
        self.zone_names = zone_names

    def update(self, zone, predicted_occupied, gt_occupied):
        if gt_occupied and predicted_occupied:
            self.TP[zone] += 1
```

```
        elif not gt_occupied and predicted_occupied:
            self.FP[zone] += 1
        elif not gt_occupied and not predicted_occupied:
            self.TN[zone] += 1
        else:
            self.FN[zone] += 1

    def report(self):
        results = {}
        for z in self.zone_names:
            tp = self.TP[z]; fp = self.FP[z]
            tn = self.TN[z]; fn = self.FN[z]
            total = tp + fp + tn + fn
            accuracy  = (tp + tn) / total if total else 0
            precision = tp / (tp + fp) if (tp + fp) else 0
            recall    = tp / (tp + fn) if (tp + fn) else 0
            f1        = (2 * precision * recall / (precision + recall)
                         if (precision + recall) else 0)
            results[z] = dict(
                accuracy=accuracy, precision=precision,
                recall=recall, f1=f1,
                TP=tp, FP=fp, TN=tn, FN=fn, total=total
            )
        return results


def save_accuracy_plots(acc_results, output_dir, violation_history, zone_names):
    os.makedirs(output_dir, exist_ok=True)

    # ── 1. Bar chart: Accuracy / Precision / Recall / F1 per zone ──────────────
    fig, ax = plt.subplots(figsize=(12, 6))
    metrics = ["accuracy", "precision", "recall", "f1"]
    colors  = ["#2196F3", "#4CAF50", "#FF9800", "#E91E63"]
    n_zones = len(zone_names)
    x       = np.arange(n_zones)
    bar_w   = 0.18
```

```
for i, (metric, color) in enumerate(zip(metrics, colors)):
    vals = [acc_results[z][metric] * 100 for z in zone_names]
    bars = ax.bar(x + i * bar_w, vals, bar_w,
                  label=metric.capitalize(), color=color, alpha=0.85)
    for bar, val in zip(bars, vals):
        ax.text(bar.get_x() + bar.get_width() / 2,
                bar.get_height() + 0.5, f"{val:.1f}",
                ha="center", va="bottom", fontsize=8, fontweight="bold")

ax.set_xticks(x + bar_w * 1.5)
ax.set_xticklabels([z[:8] for z in zone_names], fontsize=10)
ax.set_ylim(0, 112)
ax.set_ylabel("Score (%)", fontsize=12)
ax.set_title("Parking Violation Detection – Per-Zone Accuracy Metrics",
             fontsize=14, fontweight="bold")
ax.legend(loc="upper right", fontsize=10)
ax.grid(axis="y", linestyle="--", alpha=0.4)
fig.tight_layout()
out_path = os.path.join(output_dir, "accuracy_metrics.png")
fig.savefig(out_path, dpi=150)
plt.close(fig)
print(f"  [Saved] {out_path}")

# ── 2. Confusion-matrix style heat-map ──────────────────────────────────
data  = np.array([[acc_results[z]["TP"], acc_results[z]["FP"],
                   acc_results[z]["TN"], acc_results[z]["FN"]]
                  for z in zone_names], dtype=float)
fig, ax = plt.subplots(figsize=(8, 5))
im = ax.imshow(data, cmap="Blues", aspect="auto")
ax.set_xticks([0, 1, 2, 3])
ax.set_xticklabels(["TP", "FP", "TN", "FN"], fontsize=11)
ax.set_yticks(range(n_zones))
ax.set_yticklabels([z[:10] for z in zone_names], fontsize=9)
for i in range(n_zones):
    for j in range(4):
        ax.text(j, i, int(data[i, j]),
                ha="center", va="center", fontsize=11, color="black")
```

```
ax.set_title("Confusion Matrix Counts per Zone", fontsize=13, fontweight="bold")
fig.colorbar(im, ax=ax)
fig.tight_layout()
out_path = os.path.join(output_dir, "confusion_matrix.png")
fig.savefig(out_path, dpi=150)
plt.close(fig)
print(f"  [Saved] {out_path}")

if violation_history:
    frames = list(range(len(violation_history)))
    fig, ax = plt.subplots(figsize=(14, 4))
    for i, z in enumerate(zone_names):
        vals = [1 if viol_dict.get(z, False) else 0
                for viol_dict in violation_history]
        # Offset each zone slightly for clarity
        y_vals = [v * 0.8 + i for v in vals]
        color  = [c / 255 for c in ZONE_COLORS[z]]
        ax.fill_between(frames, [i] * len(frames), y_vals,
                        color=color, alpha=0.7, label=z[:8])

    ax.set_yticks(range(n_zones))
    ax.set_yticklabels([z[:10] for z in zone_names], fontsize=9)
    ax.set_xlabel("Frame Index", fontsize=11)
    ax.set_title("Violation Timeline per Zone", fontsize=13, fontweight="bold")
    ax.legend(loc="upper right", fontsize=8)
    ax.grid(axis="x", linestyle="--", alpha=0.3)
    fig.tight_layout()
    out_path = os.path.join(output_dir, "violation_timeline.png")
    fig.savefig(out_path, dpi=150)
    plt.close(fig)
    print(f"  [Saved] {out_path}")

# ── 4. Overall summary pie chart ──────────────────────────────────────

overall_tp = sum(acc_results[z]["TP"] for z in zone_names)
overall_fp = sum(acc_results[z]["FP"] for z in zone_names)
overall_tn = sum(acc_results[z]["TN"] for z in zone_names)
overall_fn = sum(acc_results[z]["FN"] for z in zone_names)
```

```
    labels = ["True Positive", "False Positive", "True Negative", "False Negative"]
    sizes  = [overall_tp, overall_fp, overall_tn, overall_fn]
    explode = (0.05, 0.05, 0.05, 0.05)
    colors_pie = ["#4CAF50", "#FF5722", "#2196F3", "#FFC107"]
    fig, ax = plt.subplots(figsize=(7, 7))
    wedges, texts, autotexts = ax.pie(
        sizes, labels=labels, autopct="%1.1f%%",
        explode=explode, colors=colors_pie, startangle=140,
        textprops={"fontsize": 11})
    for at in autotexts:
        at.set_fontweight("bold")
    ax.set_title("Overall Detection Distribution (All Zones)",
                 fontsize=13, fontweight="bold")
    fig.tight_layout()
    out_path = os.path.join(output_dir, "overall_distribution.png")
    fig.savefig(out_path, dpi=150)
    plt.close(fig)
    print(f"  [Saved] {out_path}")


def main():
    os.makedirs(OUTPUT_DIR, exist_ok=True)

    print("=" * 70)
    print("  ILLEGAL PARKING ZONE VIOLATION DETECTOR")
    print("  Algorithm: Frame Differencing + Timer-Based Violation")
    print("  Dataset  : PKLot (1k subset)")
    print("=" * 70)

    # ── Collect images ──────────────────────────────────────────────────────

    print("\n[1/5] Loading dataset …")
    image_paths = collect_images(DATASET_ROOT, limit=SUBSET_SIZE)
    if not image_paths:
        print("ERROR: No images found. Check DATASET_ROOT path.")
        sys.exit(1)
    print(f"      Found {len(image_paths)} images (using {SUBSET_SIZE} subset)")
```

```
# ── Bootstrap resolution from first frame ──────────────────────────────────
first = cv2.imread(image_paths[0])
if first is None:
    print(f"ERROR: Cannot read {image_paths[0]}")
    sys.exit(1)
IMG_H, IMG_W = first.shape[:2]
print(f"      Image resolution: {IMG_W} × {IMG_H}")

# Scale the no-parking zones to actual image size
ZONES = scale_zones(RAW_ZONES, (IMG_W, IMG_H))
zone_names = list(ZONES.keys())
print(f"      No-Parking zones: {zone_names}")

# ── Build background subtractor ──────────────────────────────────────────────

print("\n[2/5] Initialising background subtractor (MOG2) …")
bg_subtractor = cv2.createBackgroundSubtractorMOG2(
    history=BG_HISTORY,
    varThreshold=BG_THRESH,
    detectShadows=True
)

morph_kernel  = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, MORPH_KSIZE)

# ── State variables ──────────────────────────────────────────────────────────

# Per-zone timer: how many consecutive frames it has been occupied
zone_timers    = {z: 0 for z in zone_names}
# Per-zone violation flag
zone_violations = {z: False for z in zone_names}
# Per-zone "is currently occupied" (system prediction)
zone_occupied  = {z: False for z in zone_names}

acc_tracker     = AccuracyTracker(zone_names)
violation_history = []          # list of dicts {zone: bool} per frame
total_violations_triggered = 0   # cumulative count

# ── Process frames
```

```
print(f"\n[3/5] Processing {len(image_paths)} frames …")
t_start = time.time()

for frame_idx, img_path in enumerate(image_paths):
    frame = cv2.imread(img_path)
    if frame is None:
        continue

    # Resize if the image differs from the first frame (safety)
    if frame.shape[1] != IMG_W or frame.shape[0] != IMG_H:
        frame = cv2.resize(frame, (IMG_W, IMG_H))

    # ── Background subtraction ──────────────────────────────────────────
    fg_mask = bg_subtractor.apply(frame)

    # Remove shadows (shadows are grey = 127, true FG = white = 255)
    _, fg_mask = cv2.threshold(fg_mask, 200, 255, cv2.THRESH_BINARY)

    # Morphological clean-up
    fg_mask = cv2.morphologyEx(fg_mask, cv2.MORPH_OPEN,  morph_kernel)
    fg_mask = cv2.morphologyEx(fg_mask, cv2.MORPH_CLOSE, morph_kernel)
    fg_mask = cv2.dilate(fg_mask, morph_kernel, iterations=2)

    # ── Ground-truth mask using a stricter threshold ────────────────────
    # We apply a second subtractor with tighter params to get "GT"
    # Lazy initialisation: reuse the same mask with stricter threshold
    gt_fg = np.zeros_like(fg_mask)
    gt_fg[fg_mask > 0] = 255
    # Stricter: erode once more
    gt_fg = cv2.erode(gt_fg, morph_kernel, iterations=1)

    # ── Zone occupancy check ────────────────────────────────────────────
    frame_viol_state = {}
    for z_name, (x, y, w, h) in ZONES.items():
        # Clip ROI to image bounds
```

```
        x1, y1 = max(x, 0), max(y, 0)
        x2, y2 = min(x + w, IMG_W), min(y + h, IMG_H)

        fg_roi = fg_mask[y1:y2, x1:x2]
        gt_roi = gt_fg[y1:y2, x1:x2]

        pred_occ = occupancy_ratio(fg_roi) >= OCCUPANCY_THRESHOLD
        gt_occ   = occupancy_ratio(gt_roi) >= GT_THRESHOLD

        zone_occupied[z_name] = pred_occ

        # ── Timer logic ──────────────────────────────────────────────────────

        if pred_occ:
            zone_timers[z_name] += 1
        else:
            zone_timers[z_name] = 0  # reset on vacancy
            zone_violations[z_name] = False

        if zone_timers[z_name] >= VIOLATION_FRAMES and not zone_violations[z_name]:
            zone_violations[z_name] = True
            total_violations_triggered += 1

        frame_viol_state[z_name] = zone_violations[z_name]

        # ── Accumulate accuracy ──────────────────────────────────────────────

        acc_tracker.update(z_name, pred_occ, gt_occ)

    violation_history.append(dict(frame_viol_state))

    # ── Visualise ────────────────────────────────────────────────────────────

    vis = frame.copy()
    vis = draw_zones(vis, ZONES, zone_timers, zone_violations)
    vis = draw_hud(vis, frame_idx, len(image_paths), zone_violations)

    # Overlay foreground mask in corner (small thumbnail)
```

```
    thumb_h, thumb_w = IMG_H // 5, IMG_W // 5
    fg_color = cv2.cvtColor(fg_mask, cv2.COLOR_GRAY2BGR)
    thumb = cv2.resize(fg_color, (thumb_w, thumb_h))
    vis[IMG_H - thumb_h: IMG_H, IMG_W - thumb_w: IMG_W] = thumb
    cv2.putText(vis, "FG Mask", (IMG_W - thumb_w + 4, IMG_H - thumb_h + 16),
                cv2.FONT_HERSHEY_SIMPLEX, 0.45, (0, 255, 180), 1)

    # Save frame periodically
    if (frame_idx % SAVE_EVERY == 0) or (frame_idx == len(image_paths) - 1):
        save_path = os.path.join(OUTPUT_DIR, f"frame_{frame_idx:05d}.jpg")
        cv2.imwrite(save_path, vis)

    # Live display (skip if no display server)
    if SHOW_LIVE:
        disp = cv2.resize(vis, (int(IMG_W * DISPLAY_SCALE),
                                int(IMG_H * DISPLAY_SCALE)))
        try:
            cv2.imshow("Illegal Parking Zone Violation Detector", disp)
            key = cv2.waitKey(1)
            if key == 27 or key == ord("q"):   # ESC or Q to quit
                print("\n  [User quit]")
                break
        except cv2.error:
            pass  # headless environment – ignore display errors

    # Progress
    if (frame_idx + 1) % 100 == 0:
        elapsed = time.time() - t_start
        fps     = (frame_idx + 1) / elapsed
        print(f"  Frame {frame_idx+1:4d}/{len(image_paths)} | "
              f"{fps:.1f} fps | "
              f"Violations active: {sum(zone_violations.values())}")

cv2.destroyAllWindows()

elapsed_total = time.time() - t_start
avg_fps = len(image_paths) / elapsed_total
```

```
print(f"\n  Processed {len(image_paths)} frames in {elapsed_total:.1f}s "
      f"({avg_fps:.1f} fps avg)")

# ── Compute & report accuracy ──────────────────────────────────────────
print("\n[4/5] Computing accuracy metrics …")
acc_results = acc_tracker.report()

print("\n" + "─" * 70)
print(f"  {'Zone':<28} {'Acc':>7} {'Prec':>7} {'Rec':>7} {'F1':>7}  "
      f"{'TP':>5} {'FP':>5} {'TN':>5} {'FN':>5}")
print("─" * 70)

overall_acc_list = []
for z in zone_names:
    r = acc_results[z]
    overall_acc_list.append(r["accuracy"])
    print(f"  {z:<28} {r['accuracy']*100:>6.1f}% {r['precision']*100:>6.1f}% "
          f"{r['recall']*100:>6.1f}% {r['f1']*100:>6.1f}%  "
          f"{r['TP']:>5} {r['FP']:>5} {r['TN']:>5} {r['FN']:>5}")

overall_acc = float(np.mean(overall_acc_list))
print("─" * 70)
print(f"  {'OVERALL (mean)':28} {overall_acc*100:>6.1f}%")
print(f"\n  Total violation events triggered : {total_violations_triggered}")
print("─" * 70)

# ── Save plots ─────────────────────────────────────────────────────────
print("\n[5/5] Saving accuracy plots …")
save_accuracy_plots(acc_results, OUTPUT_DIR, violation_history, zone_names)

# ── Save text report ───────────────────────────────────────────────────
report_path = os.path.join(OUTPUT_DIR, "accuracy_report.txt")
with open(report_path, "w") as f:
    f.write("ILLEGAL PARKING ZONE VIOLATION DETECTION – ACCURACY REPORT\n")
    f.write("=" * 60 + "\n\n")
```

```
        f.write(f"Dataset root  : {DATASET_ROOT}\n")
        f.write(f"Frames used   : {len(image_paths)}\n")
        f.write(f"Violation rule: >= {VIOLATION_FRAMES} consecutive occupied frames\n")
        f.write(f"Detect thresh : {OCCUPANCY_THRESHOLD*100:.0f}% zone coverage\n")
        f.write(f"GT threshold  : {GT_THRESHOLD*100:.0f}% zone coverage\n\n")
        f.write(f"{'Zone':<28} {'Acc':>7} {'Prec':>7} {'Rec':>7} {'F1':>7}\n")
        f.write("-" * 60 + "\n")
        for z in zone_names:
            r = acc_results[z]
            f.write(f"{z:<28} {r['accuracy']*100:>6.1f}% {r['precision']*100:>6.1f}% "
                    f"{r['recall']*100:>6.1f}% {r['f1']*100:>6.1f}%\n")
        f.write("-" * 60 + "\n")
        f.write(f"{'OVERALL (mean)':28} {overall_acc*100:>6.1f}%\n\n")
        f.write(f"Total violation events : {total_violations_triggered}\n")

    print(f"  [Saved] {report_path}")
    print("\n✅ All done!  Results in:", OUTPUT_DIR)
    print("=" * 70)


if __name__ == "__main__":
    main()
```

APPENDIX B — SAMPLE OUTPUT SCREENSHOTS